AMS 526: Numerical Linear Algebra for Computational and Data Sciences

Lecture 20: Direct Methods for Sparse Linear Systems; Overview of Iterative Methods

Prof. Xiangmin Jiao

Stony Brook University

Outline

1. **Direct Methods for Sparse Linear Systems (MC§11.1-11.2)**
2. Overview of Iterative Methods for Sparse Linear Systems

Banded Linear Systems

- The cost of factorizing a banded linear system depends on its bandwidth
  - For an SPD $n \times n$ matrix with semi-bandwidth s, the total flop count of the Cholesky factorization is roughly ns^2
  - For an $n \times n$ matrix with lower bandwidth p and upper bandwidth q,
    - In $A = LU$ (LU without pivoting), the total flop count is about $2npq$
    - In $PA = LU$ (LU with partial pivoting), the total flop count is about $2np(p+q)$
- Banded matrices have their own special storage formats (such as Compressed Diagonal Storage (CDS))

Fill-ins

- When applying LU or Cholesky factorization to a general sparse matrix, taking linear combinations of rows or columns to annihilate unwanted nonzero entries can introduce new nonzeros into locations that were initially zero
- Such new nonzeros, called *fills* or *fill-ins*, must be stored and may themselves eventually need to be annihilated in order to obtain the triangular factors
- The resulting triangular factors can be expected to contain at least as many nonzeros as the original matrix and usually significant fill-ins as well

Sparse Cholesky Factorization

- In general, heuristic algorithms are employed to reorder the matrix to reduce fill-ins
- The amount of fill-in is sensitive to the order in which the rows and columns of the matrix are processed, so the basic problem in sparse factorization is reordering the matrix to limit fill-ins during factorization
- The exact minimization of fill-ins is a hard combinatorial problem (NP-hard in general), but heuristic algorithms such as minimum degree and nested dissection limit fill-ins well for many types of problems
- For Cholesky factorization, both rows and columns are reordered symmetrically (e.g., PAP^{T}) to preserve symmetry and positive definiteness

Graph and Sparse Matrix

$$A = \begin{bmatrix} \bullet & & & \bullet & & \bullet & \bullet & & \\ & \bullet & & & \bullet & & & \bullet & \\ & & \bullet & \bullet & \bullet & & & & \\ \bullet & & \bullet & \bullet & & \bullet & \bullet & & \bullet \\ & \bullet & \bullet & & \bullet & & & \bullet & \\ \bullet & & & \bullet & & \bullet & & & \\ \bullet & & & \bullet & & & \bullet & \bullet & \\ & \bullet & & & \bullet & & \bullet & \bullet & \\ & & \bullet & & & & & & \bullet \end{bmatrix}$$

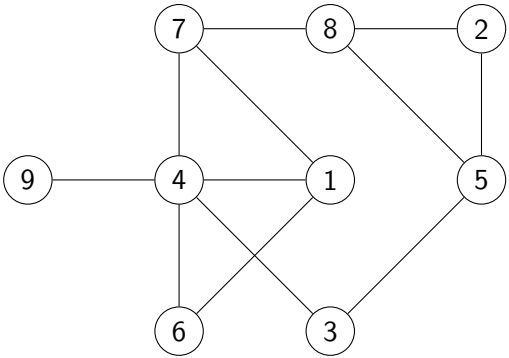


Figure: A Sparse Matrix and its Associated Graph Model

Graph Model of Elimination

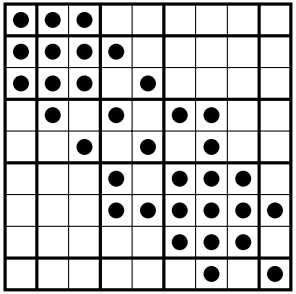
- Each step of the factorization process corresponds to the elimination of one node from the graph
- Eliminating a node causes all of its uneliminated neighboring nodes to become connected to each other
- If any such neighbors were not already connected, then a fill-in results (new edges in the graph and new nonzeros in the matrix)
- Commonly used reordering methods include Cuthill–McKee, approximate minimum degree ordering (AMD), and nested dissection

Reordering to Reduce Bandwidth

- The Cuthill–McKee algorithm and reverse Cuthill–McKee algorithm
  - The Cuthill–McKee algorithm is a variant of the breadth-first search algorithm on graphs.
    - Starts with a peripheral node
    - Generates levels R_i for $i = 1, 2, \ldots$ until all nodes are exhausted
    - The set R_{i+1} is created from the set R_i by listing all unvisited vertices adjacent to nodes in R_i
    - Within each level, nodes are listed in ascending order of their minimum predecessor (the already-visited neighbors) and, as a tiebreaker, in ascending order of their vertex degree
  - The reverse Cuthill–McKee algorithm (RCM) reverses the resulting index numbers

Example Cuthill–McKee Reordering

$$A(p,p) =$$



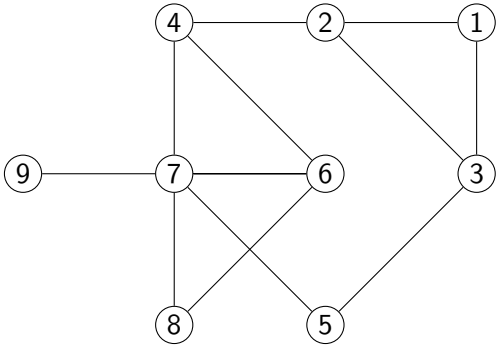
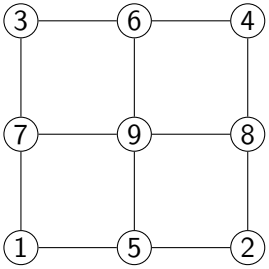


Figure: Example Cuthill–McKee Reordering

Approximate Minimum Degree Ordering

- A good heuristic for limiting fill-ins is to first eliminate those nodes having the fewest active neighbors
- The number of active neighbors is called the *degree* of a node, so the heuristic is known as *minimum degree*
- At each step, select a node of smallest degree for elimination, breaking ties arbitrarily
- After a node has been eliminated, its active neighbors become connected to each other, so the degrees of some remaining nodes may change
- The process is then repeated, with a new node of minimum degree eliminated next, and so on until all nodes have been eliminated

Minimum Degree Ordering, continued



mesh

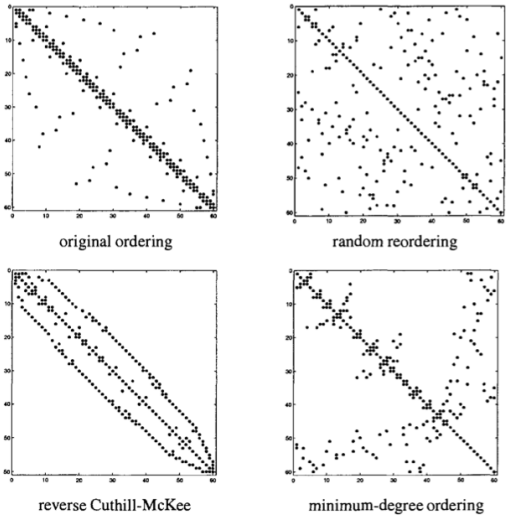
$$A = \begin{bmatrix} \times & & & & \times & & \times & \\ & \times & & & \times & & & \times \\ & & \times & & & \times & \times & \\ & & & \times & & \times & & \times \\ \times & \times & & & \times & & & \times \\ & & \times & \times & & \times & & \times \\ \times & & \times & & & & \times & \times \\ & \times & & \times & & & \times & \times \\ & & & & \times & \times & \times & \times & \times \end{bmatrix}$$

$$L = \begin{bmatrix} \times & & & & & & & \\ & \times & & & & & & \\ & & \times & & & & & \\ & & & \times & & & & \\ \times & \times & & & \times & & & \\ & & \times & \times & & \times & & \\ \times & & \times & & + & + & \times & \\ & \times & & \times & + & + & + & \times \\ & & & & \times & \times & \times & \times & \times \end{bmatrix}$$

Figure: Minimum Degree Ordering, continued

- The Cholesky factor suffers from much less fill-in than with the original natural ordering, and the advantage grows with the problem size
- Sophisticated versions of minimum degree are among the most effective general-purpose orderings known

Comparison of Different Orderings of Example Matrix



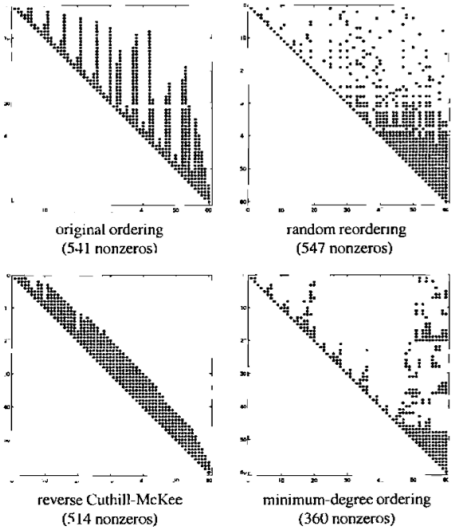


Figure: Comparison of Different Orderings. Left: Nonzero pattern of matrix A. Right: Nonzero pattern of matrix R.

Nested Dissection Ordering

- Nested dissection is based on divide-and-conquer
- First, a small set of nodes (a separator) is selected whose removal splits the graph into two disconnected pieces of roughly equal size
- No node in either piece is connected to any node in the other, so no fill-in occurs in either piece due to the elimination of any node in the other
- The separator nodes are numbered last, then the process is repeated recursively on each remaining piece of the graph until all nodes have been numbered

Nested Dissection Ordering (Cont'd)

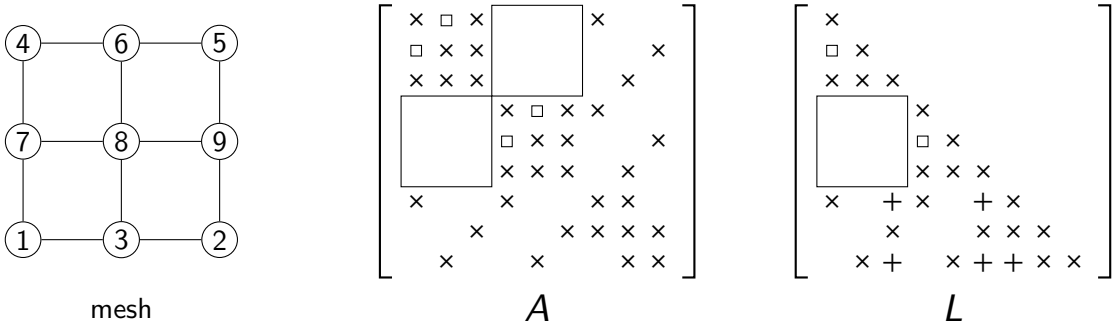


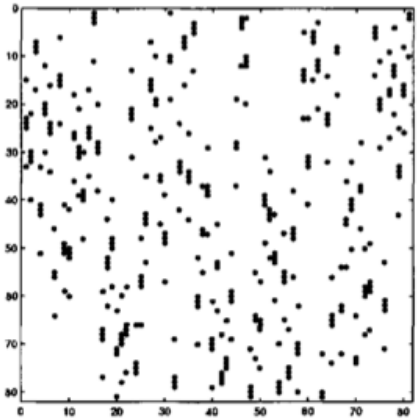
Figure: Nested Dissection Ordering

- Dissection induces blocks of zeros in the matrix that are automatically preserved during factorization
- The recursive nature of the algorithm can be seen in the hierarchical block structure of the matrix, which would involve many more levels in larger problems
- Again, the Cholesky factor suffers much less fill-in than with the original ordering, and the advantage grows with the problem size

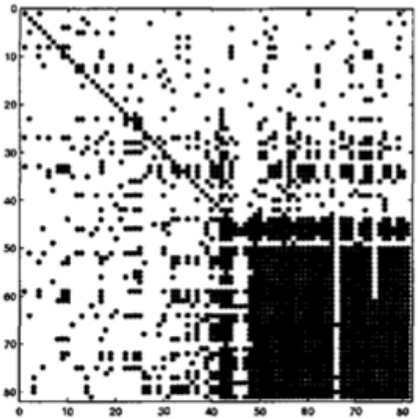
Sparse Gaussian Elimination

- For non-symmetric sparse Gaussian elimination, typically columns are statically reordered to reduce fill-ins, while rows are dynamically reordered for partial pivoting to maintain numerical stability
- Partial pivoting may introduce additional fill-ins in sparse Gaussian elimination
- Reordering may be done dynamically or statically
- The reverse Cuthill–McKee algorithm applied to the adjacency graph of $|A| + |A^{\mathsf{T}}|$ may be used to reduce the bandwidth
- *Column approximate minimum degree* (COLAMD) may be employed to reorder the matrix to reduce fill-ins

Comparison of Different Orderings of Example Matrix



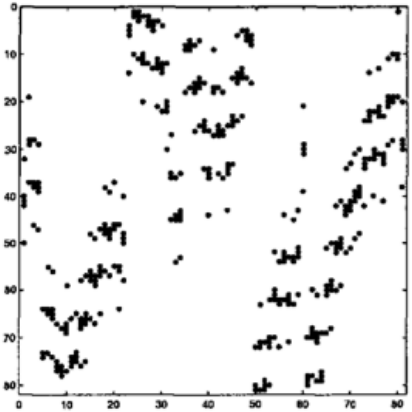
Spy plot of A



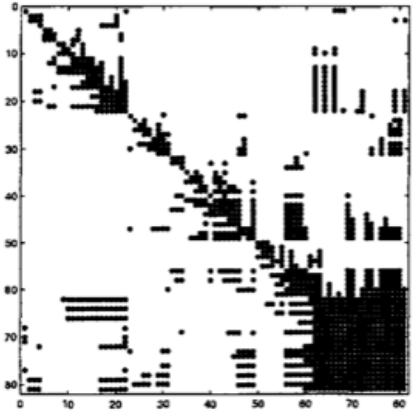
Spy plot of $L + U$
(2071 nonzeros)

Figure: Nonzero pattern of A and L+U with random ordering

Comparison of Different Orderings of Example Matrix (Cont'd)



Spy plot of A



Spy plot of $L + U$
(1249 nonzeros)

Figure: Nonzero pattern of A and L+U with column AMD ordering

Comparison of Direct Methods

- Computational cost for Laplace equation on $k \times k(\times k)$ grid with n unknowns

Method	**2-D**		**3-D**	
dense Cholesky	k^6	n^3	k^9	n^3
banded Cholesky	k^4	n^2	k^7	$n^{2.33}$
sparse Cholesky	k^3	$n^{1.5}$	k^6	n^2

Reference: Michael T. Heath, *Scientific Computing: An Introductory Survey*, 2nd Edition, McGraw-Hill, 2002.

Software of Sparse Solvers

- Additional implementation complexities include cache performance and parallelism
- It is highly advisable to use established software packages
- MATLAB has its own sparse solvers if the matrix is stored in its sparse format
  - A sparse matrix is created by using the "sparse" function
  - Reordering is implemented as "symrcm", "symamd", and "colamd"
- For symmetric matrices, an example is Taucs
- For non-symmetric matrices, popular tools include SuperLU, MUMPS, and PARDISO

Outline

1. Direct Methods for Sparse Linear Systems (MC§11.1-11.2)
2. **Overview of Iterative Methods for Sparse Linear Systems**

Direct vs. Iterative Methods

- *Direct methods*, or *noniterative methods*, compute the exact solution after a finite number of steps (assuming exact arithmetic)
  - Examples: Gaussian elimination, QR factorization
- *Iterative methods* produce a sequence of approximations $x^{(1)}, x^{(2)}, \ldots$ that hopefully converge to the true solution
  - Examples: Jacobi, Conjugate Gradient (CG), GMRES, BiCG, etc.
- Caution: The boundary between direct and iterative methods is sometimes vague (e.g., iterative methods used with direct preconditioners, or direct methods using iterative refinement)
- Why use iterative methods (instead of direct methods)?
  - may be faster than direct methods
  - produce useful intermediate results
  - handle sparse matrices more easily (need only matrix-vector products)
  - are often easier to implement on parallel computers
- **Question**: When not to use iterative methods?

Two Classes of Iterative Methods

- Stationary iterative methods are fixed-point iterations obtained by matrix splittings
  - Examples: Jacobi (for linear systems, not the Jacobi method for eigenvalues), Gauss-Seidel, Successive Over-Relaxation (SOR), etc.
- *Krylov subspace methods* find an optimal approximation in the *Krylov subspace* $\mathcal{K}_k(A, r_0) = \text{span}\{r_0, Ar_0, A^2r_0, \dots, A^{k-1}r_0\}$, where $r_0 = b - Ax_0$ is the initial residual
  - Build the subspace successively
  - Examples: Conjugate Gradient (CG), Generalized Minimum Residual (GMRES), BiCG, etc.
  - We will focus on Krylov subspace methods

Stationary Iterative Methods

- *Stationary iterative methods* find a splitting $A = M - N$ and iterate

$$x_{k+1} = M^{-1}(Nx_k + b)$$

- Suppose $r_k = b - Ax_k$ is the residual. The exact solution x_* satisfies $x_* = x_k + A^{-1}r_k$.
- A stationary iterative method approximates this by

$$x_{k+1} = x_k + M^{-1}r_k$$

because

$$\begin{aligned} x_{k+1} &= M^{-1}Nx_k + M^{-1}b \\ &= M^{-1}Nx_k + M^{-1}(r_k + Ax_k) \\ &= M^{-1}(N + A)x_k + M^{-1}r_k \\ &= x_k + M^{-1}r_k \end{aligned}$$

- A stationary iterative method converges for any initial guess if and only if the spectral radius $\rho(M^{-1}N) < 1$. It converges faster if M^{-1} is a good approximation to A^{-1}

Stationary Iterative Methods (Cont'd)

- Different choices of splittings will lead to various schemes
- Let $A = L + D + U$, where D is diagonal, L is strictly lower triangular, and U is strictly upper triangular
  - Jacobi iteration: $M = D$, converges if A is strictly or irreducibly diagonally dominant
  - Gauss-Seidel: $M = L + D$, converges if A is strictly diagonally dominant or Symmetric Positive Definite (SPD)
  - Successive Over-Relaxation (SOR): $M = \frac{1}{\omega}D + L$. For an SPD matrix A, it converges for $0 < \omega < 2$. It converges quickly with a proper choice of the relaxation parameter ω
  - Symmetric SOR (SSOR): symmetric version of SOR, often used as a preconditioner
- These methods are simple, but they may converge very slowly for some problems
- Nevertheless, stationary methods are important as preconditioners for Krylov subspace methods and as smoothers in multigrid methods (discussed later)

Stationary Iterative Methods Example

Example: For the 2D Poisson equation on an $n \times n$ interior grid, the spectral radius of the Jacobi iteration matrix is $\cos\left(\frac{\pi}{n+1}\right) \approx 1 - \mathcal{O}\left(\frac{1}{n^2}\right)$. The number of iterations required to reduce the error by a factor of ϵ is $\mathcal{O}\left(n^2 \ln \epsilon^{-1}\right)$.

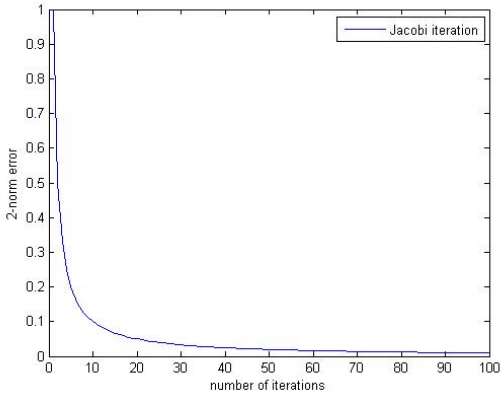


Figure: Jacobi iteration error convergence. After five Jacobi iterations on a Poisson equation, error decreases very slowly.